

Jurnal Praktikum Dasar Kecerdasan Artifisial

Modul 6: Breadth First Search (BFS)

Tujuan Praktikum

- Mahasiswa memahami dan mampu mengimplementasikan algoritma *Breadth-First Search* pada Python dengan menggunakan *library* NetworkX.

Petunjuk Umum

1. Jangan lupa untuk selalu import *library* NetworkX & Matplotlib ketika memulai sesi baru (run time) atau kode Python yang memerlukan penggunaan *library* NetworkX
2. Jangan ubah struktur kode yang diberikan, cukup isi bagian yang kosong.
3. Jalankan setiap sel satu per satu agar tidak ada error.
4. Kerjakan soal yang ada secara berurutan, dikarenakan soal akan meneruskan code soal sebelumnya

In []:

```
In [2]: import networkx as nx # Library untuk membuat graf
import matplotlib.pyplot as plt # Library untuk plot grafik
```

Fungsi pendukung untuk mencetak graf

!! Tidak usah dimodifikasi !!

```
In [3]: # Posisi node
pos = {
    'Surabaya': (4, 2),
    'Malang': (3, 0),
    'Yogyakarta': (-2, 1),
    'Semarang': (0, 3),
    'Solo': (1, 1),
    'Madiun': (1, 3),
    'Kediri': (2, 1),
    'Blitar': (2, -1),
}
```

```
In [4]: # Support function to print the graph
def show_graph(G, pos=None, title=''):
    # If position is not specified from the parameter it will generate a new pos
    if pos is None:
        # Generate spring layout if not provided
        pos = nx.spring_layout(G)
```

```

# Ukuran figure dibuat
plt.figure(figsize=(14, 6))

# Plot the graph G given in the parameter
nx.draw(G, pos, with_labels=True, node_color='green', node_size=6000,
        font_color="white", font_weight="bold", width=5)

# Retrieve each weight of each edge in graph G
edge_labels = nx.get_edge_attributes(G, "weight")

# Draw the weight of the edge in the visualization
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='pu
                             font_weight="bold", font_size=12)

# Add margins on each side
plt.margins(0.2)
# Add a title as given by the parameter
plt.title(title)
# Display the graph visualization on the terminal
plt.show()

```

1. Implementasi BFS (Arsitektur Dasar) (40)

Sari adalah seorang kurir ekspedisi yang bertugas mengirimkan paket ke berbagai kota di Pulau Jawa. Kali ini, Sari harus mengunjungi beberapa kota penting, seperti Surabaya, Malang, Yogyakarta, Semarang, Solo, Madiun, Kediri, dan Blitar. Semua kota ini dihubungkan oleh jalur pengiriman dengan jarak tertentu yang diukur dalam kilometer. Sari ingin mengetahui rute terbaik yang harus ia tempuh dari satu kota ke kota lainnya menggunakan jalur yang tersedia.

Bantu Sari untuk membuat program Python yang dapat membantu Sari dengan membuat graf dengan langkah-langkah berikut:

a. (5 poin) Inisialisasi graf bernama `jaringan` menggunakan **graf tidak berarah** untuk menunjukkan arah perjalanan.

```
In [5]: # Inisialisasi graf tidak berarah
jaringan = nx.Graph()
```

b. (7 poin) Tambahkan node untuk merepresentasikan setiap lokasi penting. Berikut ini adalah daftar node yang perlu ditambahkan ke dalam graf:

- Surabaya
- Malang
- Yogyakarta
- Semarang
- Solo
- Madiun
- Kediri
- Blitar

Pastikan semua node tersebut ditambahkan ke dalam graf sehingga setiap lokasi dapat direpresentasikan dengan baik dan nantinya dapat dihubungkan oleh edge untuk menunjukkan rute di antara lokasi-lokasi tersebut.

```
In [6]: # List daftar nama lokasi di wilayah Jawa
nodes = ["Surabaya", "Malang", "Yogyakarta", "Semarang", "Solo", "Madiun", "Kediri", "B

# Tambahkan node dari list variabel `nodes` pada graf jaringan
jaringan.add_nodes_from(nodes)

# Tampilkan graf jaringan setelah penambahan node
show_graph(jaringan, pos=pos, title="Penambahan Node pada Graf Jaringan Jawa")
```

Penambahan Node pada Graf Jaringan Jawa



Contoh output:

 Contoh output penambahan lokasi

c. (10 poin) Pada graf jaringan, setiap edge mewakili hubungan antara dua lokasi, dan masing-masing hubungan memiliki jarak tempuh tertentu yang direpresentasikan sebagai `weight` pada graf. Berikut ini adalah daftar jarak antar lokasi yang ada di wilayah tersebut:

- Surabaya ke Malang memiliki jarak 90km .
- Surabaya ke Madiun memiliki jarak 168km .
- Malang ke Blitar memiliki jarak 80km .
- Malang ke Kediri memiliki jarak 75km .
- Blitar ke Kediri memiliki jarak 45km .
- Kediri ke Solo memiliki jarak 110km .
- Madiun ke Solo memiliki jarak 112km .
- Madiun ke Semarang memiliki jarak 178km .
- Solo ke Semarang memiliki jarak 103km .
- Solo ke Yogyakarta memiliki jarak 65km .
- Semarang ke Yogyakarta memiliki jarak 125km .
- Yogyakarta ke Blitar memiliki jarak 210km .

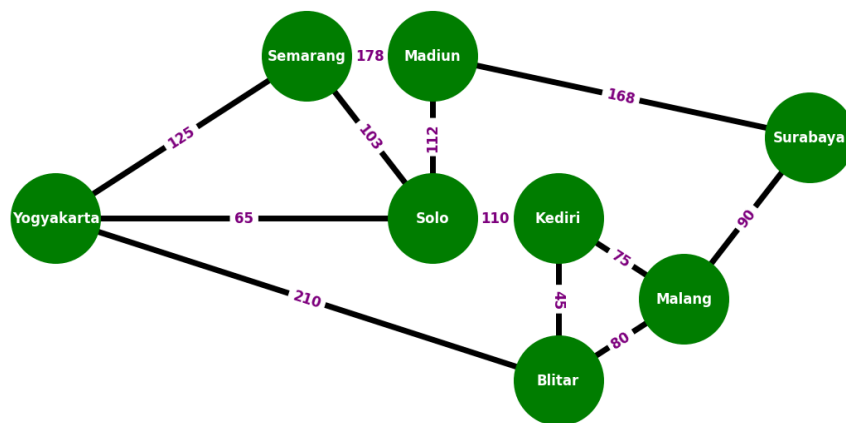
Tugas Anda adalah menambahkan edge ke dalam graf untuk setiap hubungan lokasi yang terdaftar di atas. Pastikan setiap edge mencakup informasi jarak tempuh (weight)

sebagai atribut tambahan.

```
In [7]: # Daftar hubungan (edge) antar lokasi beserta jarak tempuh (weight) masing-masing
edges = [("Surabaya", "Malang", 90),
         ("Surabaya", "Madiun", 168),
         ("Malang", "Blitar", 80),
         ("Malang", "Kediri", 75),
         ("Blitar", "Kediri", 45),
         ("Kediri", "Solo", 110),
         ("Madiun", "Solo", 112),
         ("Madiun", "Semarang", 178),
         ("Solo", "Semarang", 103),
         ("Solo", "Yogyakarta", 65),
         ("Semarang", "Yogyakarta", 125),
         ("Yogyakarta", "Blitar", 210)]

# Tambahkan edge beserta weightnya ke dalam graf jaringan
jaringan.add_weighted_edges_from(edges)

# Tampilkan graf jaringan setelah penambahan edge
show_graph(jaringan, pos=pos)
```



Contoh output:

 Contoh output penambahan lokasi

d. (10 poin) Pada potongan kode dibawah, buat program yang menampilkan informasi berikut:

- **Daftar semua nama lokasi wilayah yang terdapat pada Graf:** Program menampilkan output daftar lengkap lokasi pada graf.
- **Jarak antar lokasi:** Program ini menampilkan output setiap hubungan antara dua lokasi dan mencantumkan jarak tempuh dalam satuan kilometer.

```
In [8]: print("Elemen pada Graf jaringan")
# Menampilkan daftar semua lokasi (node) yang terdapat pada graf jaringan
print("Daftar lokasi yang terdapat pada jaringan", list(jaringan.nodes()))

print() # Menambahkan baris kosong (biarkan kosong)

# Loop melalui setiap edge pada graf untuk menampilkan informasi jarak (weight)
```

```
# Setiap edge memiliki format (asal, tujuan, data), di mana `data` menyimpan atr
for a, b, w in jaringan.edges(data=True) :
    print(f"{a} ke {b} memiliki jarak {w[\"weight\"]} km")
```

Elemen pada Graf jaringan

Daftar lokasi yang terdapat pada jaringan ['Surabaya', 'Malang', 'Yogyakarta', 'Semarang', 'Solo', 'Madiun', 'Kediri', 'Blitar']

Surabaya ke Malang memiliki jarak 90 km

Surabaya ke Madiun memiliki jarak 168 km

Malang ke Blitar memiliki jarak 80 km

Malang ke Kediri memiliki jarak 75 km

Yogyakarta ke Solo memiliki jarak 65 km

Yogyakarta ke Semarang memiliki jarak 125 km

Yogyakarta ke Blitar memiliki jarak 210 km

Semarang ke Madiun memiliki jarak 178 km

Semarang ke Solo memiliki jarak 103 km

Solo ke Kediri memiliki jarak 110 km

Solo ke Madiun memiliki jarak 112 km

Kediri ke Blitar memiliki jarak 45 km

Contoh output:

Elemen pada Graf jaringan

Daftar lokasi yang terdapat pada jaringan ['Surabaya', 'Malang', 'Yogyakarta', 'Semarang', 'Solo', 'Madiun', 'Kediri', 'Blitar']

Surabaya ke Malang memiliki jarak 90 km

Surabaya ke Madiun memiliki jarak 168 km

Malang ke Blitar memiliki jarak 80 km

Malang ke Kediri memiliki jarak 75 km

Blitar ke Kediri memiliki jarak 45 km

Kediri ke Solo memiliki jarak 110 km

Madiun ke Solo memiliki jarak 112 km

Madiun ke Semarang memiliki jarak 178 km

Solo ke Semarang memiliki jarak 103 km

Solo ke Yogyakarta memiliki jarak 65 km

Semarang ke Yogyakarta memiliki jarak 125 km

Yogyakarta ke Blitar memiliki jarak 210 km

e. (8 poin) Sari ingin mengetahui urutan perjalanan menggunakan algoritma Breadth-First Search (BFS), dimulai dari Solo sebagai titik awal. Tugas anda adalah menampilkan daftar **edge** yang dilalui oleh algoritma BFS saat menjelajahi semua lokasi di wilayah, mulai dari node Solo.

```
In [8]: print(list(nx.bfs_edges(jaringan, source="Solo"))) # menampilkan BFS dari kota S
[('Palembang', 'Batam'), ('Palembang', 'Jambi'), ('Palembang', 'Lampung'), ('Palembang', 'Bengkulu'), ('Batam', 'Medan'), ('Batam', 'Pekanbaru'), ('Jambi', 'Padang')]
```

Contoh output:

```
[('Solo', 'Kediri'), ('Solo', 'Madiun'), ('Solo', 'Semarang'),
 ('Solo', 'Yogyakarta'), ('Kediri', 'Malang'), ('Kediri',
 'Blitar'), ('Madiun', 'Surabaya'), ('Semarang', None),
 ('Yogyakarta', None)]
```

2. Implementasi BFS (Pencarian dengan Bobot) (60)

a. (10 poin) Buatlah sebuah program yang meminta input dari pengguna untuk menentukan:

- **Node awal** (`start_node`): Titik awal perjalanan Sari.
- **Node tujuan** (`end_node`): Titik tujuan yang ingin dicapai oleh Sari.

Gunakan algoritma BFS untuk menelusuri graf yang mewakili jaringan, dimulai dari `start_node` . Simpan hasil BFS dalam bentuk pasang node (`bfs_edges`) yang menunjukkan jalur pencarian dari node awal ke node-node yang terhubung.

Tampilkan hasil pencarian BFS (`bfs_edge`) yang menunjukkan semua node yang dikunjungi dan hubungan antar node yang ditemukan selama pencarian BFS dimulai dari `start_node` .

```
In [9]: def kirim_paket():
# Minta input dari pengguna untuk titik awal dan titik tujuan
start_node = input("Masukan titik awal")
end_node = input("Masukan titik tujuan")

# Temukan rute terpendek menggunakan algoritma BFS
# Menggunakan bfs_edges untuk mendapatkan jalur BFS dari node awal
bfs_edges = list(nx.bfs_edges(jaringan, source= start_node))

print(f"kota {start_node} lokasi awal perjalanan") # print start node
print(f"kota {end_node} tujuan akhir perjalanan") # print end node
print(f"Hasil pencarian BFS dari lokasi awal: {bfs_edges}") # print pencarian
```

```
In [10]: kirim_paket()
```

```
kota Palembang lokasi awal perjalanan
kota Medan tujuan akhir perjalanan
Hasil pencarian BFS dari lokasi awal: [('Palembang', 'Batam'), ('Palembang', 'Jambi'), ('Palembang', 'Lampung'), ('Palembang', 'Bengkulu'), ('Batam', 'Medan'), ('Batam', 'Pekanbaru'), ('Jambi', 'Padang')]
```

```
In [55]: kirim_paket()
```

```
kota Medan lokasi awal perjalanan
kota Lampung tujuan akhir perjalanan
Hasil pencarian BFS dari lokasi awal: [('Medan', 'Padang'), ('Medan', 'Batam'), ('Padang', 'Pekanbaru'), ('Padang', 'Jambi'), ('Batam', 'Palembang'), ('Jambi', 'Bengkulu'), ('Palembang', 'Lampung')]
```

Contoh output:

input	output
Masukan titik awal: Solo	Kota Solo lokasi awal perjalanan
Masukkan titik tujuan: Surabaya	Kota Surabaya tujuan akhir perjalanan
	Hasil pencarian BFS dari lokasi awal: [('Solo', 'Kediri'), ('Solo', 'Madiun'), ('Solo', 'Semarang'), ('Solo', 'Yogyakarta'), ('Kediri', 'Malang'), ('Kediri', 'Blitar'), ('Madiun', 'Surabaya')]
Masukan titik awal: Yogyakarta	Kota Yogyakarta lokasi awal perjalanan
Masukkan titik tujuan: Malang	Kota Malang tujuan akhir perjalanan
	Hasil pencarian BFS dari lokasi awal: [('Yogyakarta', 'Solo'), ('Yogyakarta', 'Semarang'), ('Yogyakarta', 'Blitar'), ('Solo', 'Madiun'), ('Solo', 'Kediri'), ('Blitar', 'Malang')]

b. (10 poin) Buatlah deskripsi dictionary bernama `bfs_tree` yang menyimpan hubungan parent-child dari hasil BFS. Setiap key dalam dictionary `bfs_tree` mewakili node child (tujuan), dan value-nya mewakili node parent (asal).

Gunakan pasangan node (`u` , `v`) yang diberikan oleh `bfs_edges` untuk membangun `bfs_tree` . Setiap pasangan(`u` , `v`) berarti :

- `u` adalah parent dari `v` .
- `v` adalah child dari `u` .

```
In [67]: # Buat dictionary untuk melacak jalur dari titik awal ke semua node yang dikunjungi
def pencarian_simpul_BFS(jaringan, source):
    bfs_edges = list(nx.bfs_edges(jaringan, source = source))
    print(f"Hasil BFS traversal kota {source}")
    for u, v in bfs_edges:
        print(f"{u}->{v}")
```

```
In [68]: masukan = input("Masukkan kota awal: ")
pencarian_simpul_BFS(jaringan,masukan)
```

```
Hasil BFS traversal kota Palembang
Palembang->Batam
Palembang->Jambi
Palembang->Lampung
Palembang->Bengkulu
Batam->Medan
Batam->Pekanbaru
Jambi->Padang
```

```
In [70]: masukan = input("Masukkan kota awal: ")
pencarian_simpul_BFS(jaringan,masukan)
```

```
Hasil BFS traversal kota Medan
Medan->Padang
Medan->Batam
Padang->Pekanbaru
Padang->Jambi
Batam->Palembang
Jambi->Bengkulu
Palembang->Lampung
```

Test case Program:

input	output
Masukan kota awal: Solo	Hasil BFS traversal kota Solo: Solo -> Kediri Solo -> Madiun Solo -> Semarang Solo -> Yogyakarta Kediri -> Malang Kediri -> Blitar Madiun -> Surabaya
Masukan kota awal: Yogyakarta	Hasil BFS traversal kota Yogyakarta: Yogyakarta -> Solo Yogyakarta -> Semarang Yogyakarta -> Blitar Solo -> Madiun Solo -> Kediri Blitar -> Malang Madiun -> Surabaya

c. (15 poin) Buatlah sebuah fungsi `bfs_search(bfs_tree, start, end)` yang dapat membangun jalur dari node awal (`start`) dan node tujuan (`end`) berdasarkan struktur `bfs_tree` yang diberikan.

Fungsi `bfs_search()` harus melakukan langkah-langkah berikut :

- Mulai dari `end` dan telusuri mundur ke `start` menggunakan informasi `parent-child` pada `bfs_tree`.
- Simpan jalur yang ditemukan pada list `path`
- Jika node `start` berhasil ditemukan, kembalikan `path` yang menunjukkan urutan dari node `start` ke `end`.

```
In [86]: def bfs_search(bfs_tree, start_node, end_node):
# Inisialisasi list kosong untuk menampung jalur yang ditempuh
path = []

# Mulai dari node tujuan, karena bfs_tree merepresentasikan hubungan parent-
current_node = end_node

# Telusuri jalur mundur dari node tujuan hingga mencapai node awal (start)
while current_node != start_node: # Jika node sekarang tidak sama dengan node
path.append(current_node) # Tambahkan node ke dalam jalur
current_node = bfs_tree[current_node] # Pindah ke node sebelumnya (parent)

# Tambahkan node awal setelah mencapai start
path.append(start_node)
path.reverse() # Balikkan urutan jalur agar sesuai dari start ke end menggunakan

return path
```

d. (10 poin) Buatlah sebuah fungsi `calculate_total_distance(graph, path)` yang akan menghitung total jarak tempuh dari jalur `path` pada graf `graph`. Berikut adalah langkah-langkah yang harus dilakukan dalam fungsi `calculate_total_distance()` :

- Inisialisasi variabel `total_distance` sebagai 0.

- Lakukan loop untuk mengiterasi pada setiap node pada `path` (kecuali node terakhir).
- Ambil jarak tempuh dari edge yang menghubungkan node saat ini (`node_awal`) ke node berikutnya (`node_tujuan`).
- Tambahkan weight tersebut ke dalam `total_distance` .

```
In [77]: # Fungsi untuk menghitung total jarak dari node awal ke tujuan
def calculate_total_distance(graph, path):
    total_distance = 0 # Inisialisasi jarak total

    # Loop melalui setiap node dalam path dan jumlahkan weight dari setiap edge
    for i in range(len(path) - 1):
        node_awal = path[i] # Node awal dari path
        node_tujuan = path[i+1] # Node tujuan dari path

        # Tambahkan weight dari edge ke total distance
        total_distance += graph[node_awal][node_tujuan]['weight']

    return total_distance # return total distance yang telah diperoleh
```

e. (15 poin) Sekarang, Riko ingin mengetahui rute terpendek dari lokasi awal (`kota_awal`) ke lokasi tujuan (`kota_tujuan`) berdasarkan hasil penelusuran BFS tersebut. Selain itu, Riko juga ingin mengetahui kota mana saja yang harus ia kunjungi. Selain itu, Riko juga ingin mengetahui total jarak tempuh dari jalur yang ditemukan agar ia dapat memperkirakan waktu pengiriman.

Buatlah program yang dapat menemukan rute terpendek dari node awal ke node tujuan menggunakan hasil penelusuran `bfs_tree` yang sudah dibuat dan program untuk menghitung total jarak tempuh dari jalur tersebut berdasarkan `weight` dari setiap edge yang dilalui dengan langkah-langkah berikut:

- Gunakan fungsi `bfs_search()` untuk menemukan rute dari `kota_awal` ke `kota_tujuan` .
- Jika terdapat jalur dari `kota_awal` ke `kota_tujuan` , hitung total jarak tempuh dari jalur tersebut menggunakan fungsi `calculate_total_distance()` .
- Tampilkan rute terpendek dan total jarak tempuh yang ditemukan.

```
In [84]: def calculate_shortest_city():
    kota_awal = input("Masukan titik awal") # start node
    kota_tujuan = input("Masukan titik tujuan") # end node

    bfs_edges = list(nx.bfs_edges(jaringan, source = kota_awal))

    bfs_tree = {}
    for u, v in bfs_edges:
        bfs_tree[v] = u
    # Temukan jalur dari titik awal ke titik tujuan
    shortest_path = bfs_search(bfs_tree, kota_awal, kota_tujuan)

    # Jika jalur ditemukan, hitung total jarak berdasarkan rute yang ditemukan
    if shortest_path:
        total_distance = calculate_total_distance(jaringan, shortest_path)# hitu
```

```
print("Rute: ",shortest_path) # print list dari rute terpendek
print("Total Jarak: ",total_distance ,"KM") # print total jarak tempuh
```

In [90]: `calculate_shortest_city()`

Rute: ['Padang', 'Jambi', 'Palembang', 'Lampung']
Total Jarak: 995 KM

In [92]: `calculate_shortest_city()`

Rute: ['Medan', 'Padang', 'Jambi', 'Bengkulu']
Total Jarak: 1390 KM

Test case Program:

input	output
Masukan titik awal: Yogyakarta	Rute terpendek dari Yogyakarta ke Surabaya: ['Yogyakarta', 'Solo', 'Madiun', 'Surabaya']
Masukan titik tujuan: Surabaya	Total jarak tempuh: 345 km
Masukan titik awal: Semarang	Rute terpendek dari Semarang ke Malang: ['Semarang', 'Solo', 'Kediri', 'Malang']
Masukan titik tujuan: Malang	Total jarak tempuh: 288 km